

ДИСЦИПЛИНА	Программирование в операционной системе Линукс
ИНСТИТУТ	Передовая инженерная школа СВЧ-электроники
КАФЕДРА	Передовых технологий
ВИД УЧЕБНОГО МАТЕРИАЛА	Методические указания по дисциплине
ПРЕПОДАВАТЕЛЬ	Астафьев Рустам Уралович
СЕМЕСТР	1 семестр, 2025/2026 уч. год

Ссылка на материал:

<https://github.com/astafiev-rustam/programming-in-the-linux-operating-system/tree/lecture-1-8>

Лекция №8: Создание пакетов для распространения программ

Теоретическая вводная

Философия пакетного менеджмента: от хаоса к порядку

В мире Linux распространение программного обеспечения — это не просто копирование исполняемых файлов. Это целая философия, основанная на принципах управляемости, воспроизводимости и надежности. Представьте, что вы переезжаете в новый дом. Можно просто набросать все вещи в грузовик в случайном порядке, а по приезду часами искать нужное. А можно аккуратно упаковать каждую вещь в промаркированную коробку, составить опись содержимого и расставить коробки в логическом порядке — именно так работают пакетные системы в Linux.

Пакет — это не просто архив с программой. Это интеллектуальная единица распространения, которая содержит не только файлы программы, но и метаданные о том, куда эти файлы должны быть установлены, какие другие пакеты требуются для работы (зависимости), какие действия нужно выполнить до и после установки, как корректно удалить пакет и многое другое. Это своего рода "контейнер" с инструкцией по сборке.

Две великие экосистемы: DEB и RPM

В мире Linux исторически сложились две основные системы пакетов, каждая со своей философией и инструментарией. Система DEB, рожденная в недрах Debian и унаследованная Ubuntu, основана на простых, но мощных инструментах: **dpkg** для работы с отдельными пакетами и **APT** для разрешения зависимостей. Пакеты DEB — это, по сути, архивы **ar**, содержащие два **tar**-архива: один с данными, другой с контрольной информацией.

С другой стороны, система RPM (Red Hat Package Manager) доминирует в Red Hat, CentOS, Fedora и openSUSE. RPM пакеты используют собственную бинарную структуру и обладают мощной системой скриптов, которые выполняются на разных этапах установки. Инструмент **rpm** работает с отдельными пакетами, а **yum** и его современный наследник **dnf** управляют репозиториями и разрешают зависимости.

Интересно, что обе системы в конечном счете решают одни и те же задачи, но подходят к ним с разных сторон. DEB система часто считается более простой для создания пакетов, в то время как RPM предоставляет более тонкий контроль над процессом установки.

Структура пакета: анатомия идеальной упаковки

Хороший пакет похож на хорошо организованный чемодан путешественника. Верхний слой — это контрольная информация: имя пакета, версия, архитектура, зависимости, описание. Это то, что видят менеджеры пакетов и пользователи. Под этим слоем находятся сами файлы программы, аккуратно разложенные по правильным каталогам: исполняемые файлы в `/usr/bin/`, библиотеки в `/usr/lib/`, документация в `/usr/share/doc/`, конфигурационные файлы в `/etc/`.

Но настоящая магия происходит в скриптах-помощниках: `preinst` выполняется до распаковки файлов, `postinst` — после, `prerm` — перед удалением, `postrm` — после удаления. Эти скрипты позволяют создавать пользователей, настраивать службы, обновлять конфигурации — в общем, делать все, что нужно для корректной интеграции программы в систему.

Сборка из исходников: когда готовых пакетов недостаточно

Несмотря на все удобства бинарных пакетов, иногда возникает необходимость собрать программу из исходного кода. Это может потребоваться, когда нужна самая свежая версия, не попавшая еще в репозитории, или когда требуются специфические опции конфигурации, или просто для обучения.

Классическая трилогия `./configure && make && make install` знакома каждому, кто хоть раз собирал программу в Linux. Но за этой простотой скрывается сложная система autotools, которая проверяет окружение, настраивает сборку под конкретную систему и генерирует Makefile'ы. Современные проекты все чаще используют CMake — более кроссплатформенную и мощную систему сборки.

Makefile: сердце процесса сборки

Makefile — это не просто список команд для компиляции. Это декларативное описание зависимостей между файлами и правил их преобразования. Хороший Makefile знает, что нужно пересобрать только те части проекта, которые действительно изменились, экономя время разработчика. Он определяет цели (targets), зависимости, переменные и правила — создавая тем самым карту сборки проекта.

Переменные в Makefile — это не просто подстановки, а мощный механизм настройки. `CC` определяет компилятор, `CFLAGS` — флаги компиляции, `DESTDIR` — корневой каталог для установки. Понимание этих переменных позволяет создавать гибкие и переносимые системы сборки.

Современные тенденции: контейнеры и универсальные пакеты

В последние годы традиционные системы пакетов получили развитие в виде универсальных форматов вроде Snap, Flatpak и AppImage. Эти системы решают проблему зависимости от конкретного дистрибутива, упаковывая программу вместе со всеми ее зависимостями в изолированную среду. Это похоже на переезд не с коробками, а с готовыми мебельными гарнитурами, которые можно поставить в любой квартире.

Однако традиционные пакетные системы никуда не делись — они остаются фундаментом дистрибутивов Linux, обеспечивая тесную интеграцию программ с операционной системой и эффективное использование ресурсов.

Практические примеры

Пример 1: Создание простого DEB-пакета вручную

Цель: Научиться создавать DEB-пакеты, понимая их структуру и принципы работы.

```
# 1. Создаем простую программу для упаковки
cat > hello_package.c << 'EOF'
#include <stdio.h>
#include <stdlib.h>

int main() {
    printf("=====\n");
    printf("    Hello from DEB Package!    \n");
    printf("=====\n");
    printf("This program was installed from a\n");
    printf("proper Debian package. Enjoy!    \n");
    printf("=====\n");

    // Проверяем аргументы командной строки
    printf("Program name: hello-package\n");
    printf("Version: 1.0-1\n");
    printf("Architecture: amd64\n");

    return 0;
}
EOF

# 2. Компилируем программу
gcc hello_package.c -o hello-package

# 3. Создаем структуру каталогов для пакета
mkdir -p myhello-package/DEBIAN
mkdir -p myhello-package/usr/bin
mkdir -p myhello-package/usr/share/doc/myhello-package
mkdir -p myhello-package/usr/share/man/man1

# 4. Копируем программу в нужное место
cp hello-package myhello-package/usr/bin/

# 5. Создаем файл контроля пакета - самый важный файл!
cat > myhello-package/DEBIAN/control << 'EOF'
Package: myhello-package
Version: 1.0-1
Section: utils
Priority: optional
Architecture: amd64
Depends: libc6 (>= 2.34)
Maintainer: Your Name <your.email@example.com>
Description: A simple hello world demonstration package
    This is a test package created for educational purposes.
```

It demonstrates the basic structure of a Debian package and shows how to create packages manually.

.

Features:

- * Simple hello world program
- * Proper installation to /usr/bin
- * Example documentation

Homepage: <https://example.com>

EOF

6. Создаем скрипт предустановки

```
cat > myhello-package/DEBIAN/preinst << 'EOF'
```

```
#!/bin/bash
```

```
echo "=== myhello-package Pre-Installation ==="
```

```
echo "Checking system requirements..."
```

```
# Проверяем, что система поддерживает нашу архитектуру
```

```
if [ "$(dpkg --print-architecture)" != "amd64" ]; then
```

```
    echo "Warning: This package is built for amd64 architecture"
```

```
fi
```

```
echo "Pre-installation completed successfully"
```

EOF

```
chmod 755 myhello-package/DEBIAN/preinst
```

7. Создаем скрипт постустановки

```
cat > myhello-package/DEBIAN/postinst << 'EOF'
```

```
#!/bin/bash
```

```
echo "=== myhello-package Post-Installation ==="
```

```
echo "The package has been successfully installed!"
```

```
echo "You can now run 'hello-package' from anywhere in the terminal."
```

```
echo "Post-installation completed successfully"
```

EOF

```
chmod 755 myhello-package/DEBIAN/postinst
```

8. Создаем скрипт предудаления

```
cat > myhello-package/DEBIAN/prerm << 'EOF'
```

```
#!/bin/bash
```

```
echo "=== myhello-package Pre-Removal ==="
```

```
echo "Preparing to remove myhello-package..."
```

```
echo "Pre-removal completed successfully"
```

EOF

```
chmod 755 myhello-package/DEBIAN/prerm
```

9. Создаем скрипт постудаления

```
cat > myhello-package/DEBIAN/postrm << 'EOF'
```

```
#!/bin/bash
```

```
echo "=== myhello-package Post-Removal ==="
```

```
echo "Package myhello-package has been completely removed."
```

```
echo "Thank you for using our software!"
```

EOF

```
chmod 755 myhello-package/DEBIAN/postrm
```

10. Добавляем документацию

```
cat > myhello-package/usr/share/doc/myhello-package/README << 'EOF'
```

```
MyHello Package
```

=====

This is a demonstration package created for educational purposes.

Installation:

The package installs a single executable: hello-package

Usage:

Simply run: hello-package

Removal:

To remove: `sudo dpkg -r myhello-package`

License:

This is free software. Use at your own risk.

EOF

11. Добавляем man-страницу

`cat > myhello-package/usr/share/man/man1/hello-package.1 << 'EOF'`

`.TH HELLO-PACKAGE 1 "2024-01-01" "1.0-1" "User Commands"`

`.SH NAME`

`hello-package \- a simple hello world demonstration program`

`.SH SYNOPSIS`

`.B hello-package`

`.SH DESCRIPTION`

`.B hello-package`

`is a demonstration program that shows a friendly greeting message.`

`It was created to demonstrate Debian package creation.`

`.SH OPTIONS`

`This program does not accept any command-line options.`

`.SH AUTHOR`

`Your Name <your.email@example.com>`

`.SH "SEE ALSO"`

`.BR echo (1)`

`EOF`

`gzip -9 myhello-package/usr/share/man/man1/hello-package.1`

12. Создаем файл авторских прав

`cat > myhello-package/usr/share/doc/myhello-package/copyright << 'EOF'`

`Format: https://www.debian.org/doc/packaging-manuals/copyright-format/1.0/`

`Upstream-Name: myhello-package`

`Source: https://example.com`

`Files: *`

`Copyright: 2024 Your Name <your.email@example.com>`

`License: MIT`

`License: MIT`

`Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal`

in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

- The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

- THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

EOF

13. Собираем пакет

```
dpkg-deb --build myhello-package
```

14. Проверяем созданный пакет

```
echo "=== Информация о созданном пакете ==="
dpkg --info myhello-package.deb
```

```
echo -e "\n=== Содержимое пакета ==="
dpkg --contents myhello-package.deb
```

15. Устанавливаем пакет

```
echo -e "\n=== Установка пакета ==="
sudo dpkg -i myhello-package.deb
```

16. Проверяем установку

```
echo -e "\n=== Проверка установки ==="
hello-package
which hello-package
```

17. Проверяем информацию об установленном пакете

```
echo -e "\n=== Информация об установленном пакете ==="
dpkg -l myhello-package
dpkg -L myhello-package
```

18. Удаляем пакет

```
echo -e "\n=== Удаление пакета ==="
sudo dpkg -r myhello-package
```

19. Проверяем удаление

```
echo -e "\n=== Проверка удаления ==="
which hello-package || echo "Программа успешно удалена"
```

Пример 2: Создание RPM пакета с использованием срес файла

Цель: Освоить создание RPM пакетов с помощью срес файлов.

```
# 1. Устанавливаем необходимые инструменты для RPM
sudo apt install rpm || echo "RPM tools not available, continuing with
simulation..."

# 2. Создаем программу для упаковки
cat > rpm-demo.c << 'EOF'
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    printf("=== RPM Package Demonstration ===\n\n");

    if (argc == 1) {
        printf("Usage: %s <name>\n", argv[0]);
        printf("Example: %s Linux\n", argv[0]);
        return 1;
    }

    printf("Hello %s from RPM package!\n", argv[1]);
    printf("\nPackage Information:\n");
    printf("  Name: rpm-demo\n");
    printf("  Version: 1.0\n");
    printf("  Release: 1\n");
    printf("  Architecture: x86_64\n");

    return 0;
}
EOF

# 3. Компилируем программу
gcc rpm-demo.c -o rpm-demo

# 4. Создаем структуру для сборки RPM
mkdir -p rpm-build/{BUILD,RPMS,SOURCES,SPECS,SRPMS,tmp}

# 5. Создаем архив с исходным кодом
tar -czvf rpm-build/SOURCES/rpm-demo-1.0.tar.gz rpm-demo

# 6. Создаем spec файл - сердце RPM пакета
cat > rpm-build/SPECS/rpm-demo.spec << 'EOF'
Name:                rpm-demo
Version:              1.0
Release:              1%{?dist}
Summary:              A demonstration RPM package

Group:                Applications/System
License:              MIT
URL:                  https://example.com
Source0:              %{name}-%{version}.tar.gz
BuildRoot:            %{_tmppath}/%{name}-%{version}-%{release}-root-%(%{__id_u} -n)

BuildArch:            x86_64
Requires:              glibc
```

```
%description
This is a demonstration RPM package created for educational purposes.
It shows how to create proper RPM packages with all necessary components.

%prep
%setup -q

%build
# В реальном пакете здесь была бы компиляция
echo "Building the package..."

%install
rm -rf %{buildroot}
mkdir -p %{buildroot}/%{_bindir}
mkdir -p %{buildroot}/%{_mandir}/man1
mkdir -p %{buildroot}/%{_docdir}/%{name}

# Устанавливаем программу
install -m 755 rpm-demo %{buildroot}/%{_bindir}

# Создаем man страницу
cat > %{buildroot}/%{_mandir}/man1/rpm-demo.1 << 'MANPAGE'
.TH RPM-DEMO 1 "2024-01-01" "1.0" "User Commands"
.SH NAME
rpm-demo \- a demonstration RPM package
.SH SYNOPSIS
.B rpm-demo
.I name
.SH DESCRIPTION
.B rpm-demo
is a demonstration program that shows a personalized greeting.
.SH OPTIONS
.I name
The name to display in the greeting.
.SH EXAMPLES
.TP
.B rpm-demo Linux
Displays "Hello Linux from RPM package!"
.SH AUTHOR
Your Name <your.email@example.com>
MANPAGE

# Создаем документацию
cat > %{buildroot}/%{_docdir}/%{name}/README << 'DOCUMENTATION'
RPM Demo Package
=====

This is a demonstration RPM package.

Features:
-----
* Simple greeting program
* Proper RPM packaging
```



```
* Man page included

Usage:
-----
rpm-demo <name>

Documentation:
-----
man rpm-demo
DOCUMENTATION

%clean
rm -rf %{buildroot}

%files
%defattr(-,root,root,-)
%{_bindir}/rpm-demo
%{_mandir}/man1/rpm-demo.1*
%doc %{_docdir}/%{name}

%changelog
* Tue Jan 01 2024 Your Name <your.email@example.com> - 1.0-1
- Initial package creation

%pre
echo "Pre-installation script running..."
echo "Installing rpm-demo version %{version}-%{release}"

%post
echo "Post-installation script running..."
echo "rpm-demo %{version}-%{release} successfully installed!"

%preun
echo "Pre-uninstall script running..."
echo "Removing rpm-demo %{version}-%{release}"

%postun
echo "Post-uninstall script running..."
echo "rpm-demo %{version}-%{release} successfully removed!"
EOF

# 7. В реальной системе мы бы собрали пакет так:
# rpmbuild --define "_topdir $(pwd)/rpm-build" -bb rpm-build/SPECS/rpm-demo.spec

# 8. Для демонстрации создаем имитацию RPM пакета
echo "=== Создание RPM пакета (имитация) ==="
mkdir -p rpm-package/RPMS/x86_64
cat > rpm-package/RPMS/x86_64/rpm-demo-1.0-1.x86_64.rpm << 'EOF'
This would be a real RPM package file
containing the compiled program and all
necessary metadata and scripts.
EOF

# 9. Создаем альтернативный способ для Debian/Ubuntu систем
```

```
echo "=== Альтернативная демонстрация для DEB-систем ==="

# Создаем DEB пакет аналогичный RPM
mkdir -p rpm-demo-pkg/DEBIAN
mkdir -p rpm-demo-pkg/usr/bin
mkdir -p rpm-demo-pkg/usr/share/man/man1
mkdir -p rpm-demo-pkg/usr/share/doc/rpm-demo

cp rpm-demo rpm-demo-pkg/usr/bin/

cat > rpm-demo-pkg/DEBIAN/control << 'EOF'
Package: rpm-demo
Version: 1.0-1
Architecture: amd64
Maintainer: Your Name <your.email@example.com>
Depends: libc6 (>= 2.34)
Section: utils
Priority: optional
Description: RPM-style demonstration package for DEB systems
  This package demonstrates RPM-like packaging in DEB format.
  It shows cross-packaging concepts and techniques.
EOF

# Создаем man страницу
cat > rpm-demo-pkg/usr/share/man/man1/rpm-demo.1 << 'EOF'
.TH RPM-DEMO 1 "2024-01-01" "1.0" "User Commands"
.SH NAME
rpm-demo \- demonstration package with RPM-style packaging
.SH SYNOPSIS
.B rpm-demo
.I name
.SH DESCRIPTION
Demonstration package showing packaging concepts.
.SH AUTHOR
Your Name <your.email@example.com>
EOF
gzip -9 rpm-demo-pkg/usr/share/man/man1/rpm-demo.1

# Собираем пакет
dpkg-deb --build rpm-demo-pkg

# 10. Демонстрируем установку и использование
echo -e "\n=== Установка и тестирование ==="
sudo dpkg -i rpm-demo-pkg.deb

echo -e "\n=== Тестирование программы ==="
rpm-demo "RPM Packaging"

echo -e "\n=== Проверка установленных файлов ==="
dpkg -L rpm-demo

# 11. Показываем различия между пакетными системами
echo -e "\n=== Сравнение пакетных систем ==="
echo "DEB пакеты:"
```

```
echo " - Используют dpkg/APT"
echo " - Файлы: control, preinst, postinst, prerm, postrm"
echo " - Структура: ar архив с двумя tar архивами"
echo ""
echo "RPM пакеты:"
echo " - Используют rpm/yum/dnf"
echo " - Файлы: .spec файл с секциями"
echo " - Структура: srp0 архив с заголовком"
echo ""
echo "Общие концепции:"
echo " - Метаданные (имя, версия, зависимости)"
echo " - Скрипты установки/удаления"
echo " - Управление файлами"
echo " - Проверки зависимостей"

# 12. Очистка
sudo dpkg -r rpm-demo
```

Пример 3: Профессиональный Makefile для сборки и упаковки

Цель: Создать продвинутый Makefile для автоматизации сборки и упаковки.

```
# 1. Создаем многофайловый проект для демонстрации
mkdir -p myproject/src
mkdir -p myproject/include
mkdir -p myproject/docs

# 2. Создаем заголовочные файлы
cat > myproject/include/utils.h << 'EOF'
#ifndef UTILS_H
#define UTILS_H

void print_banner(void);
int calculate_sum(int a, int b);
void show_version(void);

#endif
EOF

cat > myproject/include/config.h << 'EOF'
#ifndef CONFIG_H
#define CONFIG_H

#define PACKAGE_NAME "MyProject"
#define PACKAGE_VERSION "1.2.3"
#define PACKAGE_BUGREPORT "bugs@example.com"

#endif
EOF

# 3. Создаем исходные файлы
```

```
cat > myproject/src/utils.c << 'EOF'
#include <stdio.h>
#include "utils.h"
#include "config.h"

void print_banner(void) {
    printf("=== %s ===\n", PACKAGE_NAME);
    printf("Version: %s\n", PACKAGE_VERSION);
    printf("=====\n");
}

int calculate_sum(int a, int b) {
    return a + b;
}

void show_version(void) {
    printf("%s version %s\n", PACKAGE_NAME, PACKAGE_VERSION);
    printf("Report bugs to: %s\n", PACKAGE_BUGREPORT);
}
EOF

cat > myproject/src/main.c << 'EOF'
#include <stdio.h>
#include <stdlib.h>
#include "utils.h"
#include "config.h"

int main(int argc, char *argv[]) {
    if (argc < 2) {
        print_banner();
        printf("Usage: %s <command>\n", argv[0]);
        printf("Commands: sum, version\n");
        return 1;
    }

    if (strcmp(argv[1], "sum") == 0) {
        if (argc != 4) {
            printf("Usage: %s sum <num1> <num2>\n", argv[0]);
            return 1;
        }
        int a = atoi(argv[2]);
        int b = atoi(argv[3]);
        printf("Sum: %d + %d = %d\n", a, b, calculate_sum(a, b));
    }
    else if (strcmp(argv[1], "version") == 0) {
        show_version();
    }
    else {
        printf("Unknown command: %s\n", argv[1]);
        return 1;
    }

    return 0;
}
```

EOF

```
# 4. Создаем профессиональный Makefile
cat > myproject/Makefile << 'EOF'
# Professional Makefile for Project Build and Packaging
# =====

# Project Configuration
PACKAGE_NAME = myproject
PACKAGE_VERSION = 1.2.3
PACKAGE_RELEASE = 1
ARCHITECTURE = amd64

# Compiler and Flags
CC = gcc
CFLAGS = -Wall -Wextra -Wpedantic -g -I./include
LDFLAGS =
DEBUG_CFLAGS = -DDEBUG -O0
RELEASE_CFLAGS = -O2 -DNDEBUG

# Directories
SRCDIR = src
INCDIR = include
BUILDDIR = build
BINDIR = $(BUILDDIR)/bin
OBJDIR = $(BUILDDIR)/obj
PKGDIR = $(BUILDDIR)/pkg
DOCSDIR = docs

# Targets
TARGET = $(BINDIR)/$(PACKAGE_NAME)
SOURCES = $(wildcard $(SRCDIR)/*.c)
OBJECTS = $(SOURCES:$(SRCDIR)/%.c=$(OBJDIR)/%.o)

# Default target
all: debug

# Debug build
debug: CFLAGS += $(DEBUG_CFLAGS)
debug: $(TARGET)

# Release build
release: CFLAGS += $(RELEASE_CFLAGS)
release: $(TARGET)

# Create target executable
$(TARGET): $(OBJECTS) | $(BINDIR)
    $(CC) $(OBJECTS) -o $@ $(LDFLAGS)
    @echo "Built target: $@"

# Compile source files
$(OBJDIR)/%.o: $(SRCDIR)/%.c | $(OBJDIR)
    $(CC) $(CFLAGS) -c $< -o $@
```

```
# Create directories
$(BINDIR):
    @mkdir -p $(BINDIR)

$(OBJDIR):
    @mkdir -p $(OBJDIR)

$(PKGDIR):
    @mkdir -p $(PKGDIR)

# Installation
PREFIX = /usr/local
BIN_INSTALL_DIR = $(DESTDIR)$(PREFIX)/bin
DOC_INSTALL_DIR = $(DESTDIR)$(PREFIX)/share/doc/$(PACKAGE_NAME)
MAN_INSTALL_DIR = $(DESTDIR)$(PREFIX)/share/man/man1

install: release
    @echo "Installing $(PACKAGE_NAME) to $(PREFIX)"
    install -d $(BIN_INSTALL_DIR)
    install -m 755 $(TARGET) $(BIN_INSTALL_DIR)/
    install -d $(DOC_INSTALL_DIR)
    install -m 644 $(DOCS_DIR)/* $(DOC_INSTALL_DIR)/ 2>/dev/null || true
    @echo "Installation completed"

uninstall:
    @echo "Uninstalling $(PACKAGE_NAME)"
    rm -f $(BIN_INSTALL_DIR)/$(PACKAGE_NAME)
    rm -rf $(DOC_INSTALL_DIR)
    @echo "Uninstallation completed"

# Package creation
DEB_PKG_DIR =
$(PKGDIR)/$(PACKAGE_NAME)_$(PACKAGE_VERSION)-$(PACKAGE_RELEASE)_$(ARCHITECTURE)

deb: release
    @echo "Building DEB package..."
    @mkdir -p $(DEB_PKG_DIR)/DEBIAN
    @mkdir -p $(DEB_PKG_DIR)/usr/bin
    @mkdir -p $(DEB_PKG_DIR)/usr/share/doc/$(PACKAGE_NAME)

    # Copy binary
    cp $(TARGET) $(DEB_PKG_DIR)/usr/bin/

    # Create control file
    @cat > $(DEB_PKG_DIR)/DEBIAN/control << CONTROL_EOF
Package: $(PACKAGE_NAME)
Version: $(PACKAGE_VERSION)-$(PACKAGE_RELEASE)
Section: utils
Priority: optional
Architecture: $(ARCHITECTURE)
Depends: libc6 (>= 2.34)
Maintainer: Package Maintainer <maintainer@example.com>
Description: Professional example package built with Makefile
    This package demonstrates professional build and packaging
```

```

techniques using Makefile automation.
CONTROL_EOF

# Create postinst script
@cat > $(DEB_PKG_DIR)/DEBIAN/postinst << SCRIPT_EOF
#!/bin/bash
echo "$(PACKAGE_NAME) version $(PACKAGE_VERSION) has been installed"
SCRIPT_EOF
chmod 755 $(DEB_PKG_DIR)/DEBIAN/postinst

# Build package
dpkg-deb --build $(DEB_PKG_DIR)
@echo "DEB package created: $(DEB_PKG_DIR).deb"

# Testing
test: debug
    @echo "Running tests..."
    $(TARGET) version
    $(TARGET) sum 5 7
    @echo "Tests completed"

# Code quality
check:
    @echo "Running code quality checks..."
    @echo "Checking for TODO comments..."
    @grep -r "TODO" $(SRCDIR) $(INCDIR) || true
    @echo "Code quality check completed"

# Distribution package
dist: clean deb
    @echo "Creating distribution package..."
    tar -czf $(PACKAGE_NAME)-$(PACKAGE_VERSION).tar.gz \
        --transform 's,^,$(PACKAGE_NAME)-$(PACKAGE_VERSION)/,' \
        $(SRCDIR) $(INCDIR) $(DOCS_DIR) Makefile README.md
    @echo "Distribution package created:
$(PACKAGE_NAME)-$(PACKAGE_VERSION).tar.gz"

# Cleanup
clean:
    @echo "Cleaning build files..."
    rm -rf $(BUILDDIR)
    rm -f $(PACKAGE_NAME)-*.tar.gz

distclean: clean
    @echo "Deep cleaning..."
    rm -f *.deb

# Help
help:
    @echo "Available targets:"
    @echo "  all          - Build debug version (default)"
    @echo "  debug       - Build with debug flags"
    @echo "  release     - Build with optimization"
    @echo "  install     - Install to system"

```

```

@echo "  uninstall - Remove from system"
@echo "  deb       - Create DEB package"
@echo "  test       - Run basic tests"
@echo "  check      - Code quality checks"
@echo "  dist       - Create distribution package"
@echo "  clean      - Remove build files"
@echo "  distclean  - Remove all generated files"
@echo "  help       - Show this help"

# PHONY targets
.PHONY: all debug release install uninstall deb test check dist clean distclean
help

# Dependency information
-include $(OBJECTS:.o=.d)

$(OBJDIR)/%.o: $(SRCDIR)/%.c | $(OBJDIR)
    $(CC) $(CFLAGS) -c $< -o $@
    @$ (CC) -MM $(CFLAGS) $< > $(OBJDIR)/$*.d
    @mv -f $(OBJDIR)/$*.d $(OBJDIR)/$*.d.tmp
    @sed -e 's|.|$@:|' < $(OBJDIR)/$*.d.tmp > $(OBJDIR)/$*.d
    @sed -e 's/./:/' -e 's/\\$$//' < $(OBJDIR)/$*.d.tmp | fmt -1 | \
        sed -e 's/^ */' -e 's/$$/:/' >> $(OBJDIR)/$*.d
    @rm -f $(OBJDIR)/$*.d.tmp
EOF

# 5. Создаем документацию
cat > myproject/README.md << 'EOF'
# MyProject

Professional example project with automated build and packaging.

## Building

make          # Debug build
make release  # Release build
make deb      # Create DEB package

## Installation

make install

## Packaging

The Makefile supports creating DEB packages and distribution tarballs.
EOF

cat > myproject/docs/API.md << 'EOF'
# API Documentation

## Functions

### print_banner()
Prints the application banner.

```



```
### calculate_sum(a, b)
Calculates the sum of two numbers.

### show_version()
Displays version information.
EOF

# 6. Демонстрируем использование Makefile
cd myproject

echo "=== Демонстрация профессионального Makefile ==="

echo -e "\n1. Сборка debug версии:"
make debug

echo -e "\n2. Запуск тестов:"
make test

echo -e "\n3. Проверка качества кода:"
make check

echo -e "\n4. Сборка release версии:"
make release

echo -e "\n5. Создание DEB пакета:"
make deb

echo -e "\n6. Просмотр помощи:"
make help

echo -e "\n7. Создание дистрибутивного пакета:"
make dist

echo -e "\n8. Очистка:"
make clean

cd ..

# 7. Показываем созданные артефакты
echo -e "\n=== Созданные артефакты ==="
find myproject -name "*.deb" -o -name "*.tar.gz" | head -10
```

Пример 4: Пакет с системным сервисом

```
# Создаем демон-программу
cat > myservice.c << 'EOF'
#include <stdio.h>
#include <unistd.h>
#include <signal.h>
```

```
volatile int running = 1;

void handle_signal(int sig) {
    running = 0;
}

int main() {
    signal(SIGTERM, handle_signal);

    while (running) {
        printf("Service is running...\n");
        sleep(5);
    }

    printf("Service stopped gracefully\n");
    return 0;
}
EOF
gcc myservice.c -o myservice

# Создаем systemd service файл
cat > myservice.service << 'EOF'
[Unit]
Description=My Demo Service
After=network.target

[Service]
Type=simple
ExecStart=/usr/bin/myservice
Restart=always

[Install]
WantedBy=multi-user.target
EOF

# Создаем пакет
mkdir -p service-pkg/DEBIAN
mkdir -p service-pkg/usr/bin
mkdir -p service-pkg/etc/systemd/system

cp myservice service-pkg/usr/bin/
cp myservice.service service-pkg/etc/systemd/system/

cat > service-pkg/DEBIAN/control << 'EOF'
Package: myservice
Version: 1.0-1
Architecture: amd64
Maintainer: SysAdmin <admin@example.com>
Description: System service example
    Demonstrates service packaging.
EOF

# Скрипт постустановки
cat > service-pkg/DEBIAN/postinst << 'EOF'
```

```
#!/bin/bash
systemctl daemon-reload
systemctl enable myservice.service
echo "Service installed and enabled"
EOF
chmod +x service-pkg/DEBIAN/postinst

# Скрипт предудаления
cat > service-pkg/DEBIAN/prerm << 'EOF'
#!/bin/bash
systemctl stop myservice.service
systemctl disable myservice.service
echo "Service stopped and disabled"
EOF
chmod +x service-pkg/DEBIAN/prerm

dpkg-deb --build service-pkg

# Проверяем содержимое пакета
dpkg --contents service-pkg.deb
```

Пример 5: Пакет с конфигурационными файлами

```
# Создаем программу, читающую конфиг
cat > config-app.c << 'EOF'
#include <stdio.h>
#include <stdlib.h>

int main() {
    FILE *config = fopen("/etc/myapp/config.conf", "r");
    if (config) {
        char line[256];
        printf("Configuration:\n");
        while (fgets(line, sizeof(line), config)) {
            printf("  %s", line);
        }
        fclose(config);
    } else {
        printf("Using default configuration\n");
    }
    return 0;
}
EOF
gcc config-app.c -o config-app

# Создаем конфигурационный файл
mkdir -p etc/myapp
cat > etc/myapp/config.conf << 'EOF'
# MyApp Configuration
server = localhost
port = 8080
```

```
timeout = 30
EOF

# Создаем пакет
mkdir -p config-pkg/DEBIAN
mkdir -p config-pkg/usr/bin
mkdir -p config-pkg/etc/myapp

cp config-app config-pkg/usr/bin/
cp etc/myapp/config.conf config-pkg/etc/myapp/

cat > config-pkg/DEBIAN/control << 'EOF'
Package: config-app
Version: 1.0-1
Architecture: amd64
Maintainer: Config Master <config@example.com>
Description: Configuration example
  Shows how to package config files.
EOF

# Помечаем конфиг файлы
cat > config-pkg/DEBIAN/conffiles << 'EOF'
/etc/myapp/config.conf
EOF

dpkg-deb --build config-pkg

# Устанавливаем и проверяем
sudo dpkg -i config-pkg.deb
config-app

# Показываем, что конфиг сохранился при обновлении
sudo dpkg -r config-app
sudo dpkg -i config-pkg.deb

# Очистка
sudo dpkg -r config-app
```